LOW-LEVEL DESIGN (LLD)

RAG-Based Customer Support Assistant using LangGraph and HITL

Module-Level Design

The system is divided into multiple functional modules to ensure modular workflow execution and efficient query processing.

- **Document Processing Module**
- This module loads the PDF knowledge base using PyPDFLoader and extracts textual data for preprocessing.
- **Chunking Module**
- This module splits extracted text into overlapping segments of 500 tokens with 50-token overlap to maintain semantic continuity.
- **Embedding Module**
- This module converts document chunks into vector embeddings using Sentence Transformers for similarity-based retrieval.
- **Vector Storage Module**
- This module stores generated embeddings inside ChromaDB for efficient indexing and retrieval.
- **Retrieval Module**
- This module performs similarity search on stored embeddings and retrieves the most relevant chunks corresponding to user queries.
- **Query Processing Module**
- This module processes user queries by converting them into embeddings and retrieving contextual information from ChromaDB.
- **Graph Execution Module**
- This module manages workflow orchestration using LangGraph and ensures sequential execution between nodes.
- **HITL Module**
- This module handles escalation of low-confidence or complex queries to a human support agent.

Data Structures

The system uses structured data objects to maintain workflow state and retrieval efficiency.

- **Document Object**
- Stores extracted PDF content before chunking.
- **Chunk Object**
- Stores segmented text along with metadata such as chunk index and source location.
- **Embedding Vector**

- Stores numerical vector representation of document chunks.
- **Query-Response Schema**
- Stores user query, retrieved chunks, generated response, and confidence score.
- **Graph State Object**
- Maintains workflow execution state across LangGraph nodes.

Workflow Design Using LangGraph

The workflow execution is controlled using LangGraph nodes and edges.

Nodes

- Input Node receives user query
- Processing Node retrieves document chunks
- Decision Node evaluates response confidence
- Output Node generates final response
- HITL Node escalates complex queries

Conditional Routing Logic

Routing decisions are based on response confidence and query complexity.

Automatic response is generated when similarity score exceeds threshold value.

Escalation is triggered when:

- similarity score is low
- context is missing
- query is outside knowledge base
- user explicitly requests human support

HITL Design

- The Human-in-the-Loop module ensures reliability in handling complex queries.
- Escalation occurs when response confidence falls below threshold level.
- The system forwards the query to a human agent interface.
- The agent reviews the query and provides a final response.
- The response is returned to the user through the output node.

API / Interface Design

The system supports a command-line interface for user interaction.

Input Format

User enters query through CLI prompt.

Example:

What is the refund policy?

Output Format

System returns contextual response retrieved from PDF knowledge base or escalated human response.

Error Handling

The system includes multiple error-handling mechanisms to ensure stability.

- Missing PDF file detection
- Empty retrieval results handling
- Embedding generation failure handling
- Low-confidence response fallback
- Workflow execution interruption handling